

# Untitled

October 17, 2025

# Project Assignment 3 ##### Tisha Laiwalla

## 0.1 Modeling and Visualizing the Crime Analysis in London

### 0.1.1 1. Introduction

This project continues the geospatial analysis of crime in London, focusing on analysis, modeling, and simulation. Using street-level crime data from 2024 to:

- Identify crime hotspots using clustering techniques.
- Model temporal trends in crime frequency.
- Simulate crime density using kernel density estimation.
- Visualize results using interactive maps and static plots.

#### Imports:

```
[1]: import pandas as pd
import numpy as np
import geopandas as gpd
%matplotlib inline
import os
from datetime import datetime
os.environ["PROJ_LIB"] = r'/opt/conda/pkgs/proj4-5.2.0-he1b5a44_1006/share/proj/
↵
from geopandas import GeoSeries, GeoDataFrame
from shapely.geometry import Point
import mapclassify
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import cartopy.feature as cf
import json
import folium
import mplleaflet
import cartopy.io.shapereader as shpreader
from shapely.geometry import shape
import fiona
import IPython
from pandas import Series
```

## 0.1.2 2. Data Loading and Preprocessing

```
[2]: df1 = pd.read_csv('London_Crimes_2024.csv')
df1 = df1.drop(columns=['Crime ID'])
df1
```

```
[2]:      Month      Reported by      Crime type \
0      2024-01  Metropolitan Police Service      Other theft
1      2024-01  Metropolitan Police Service  Violence and sexual offences
2      2024-01  Metropolitan Police Service  Violence and sexual offences
3      2024-01  Metropolitan Police Service  Violence and sexual offences
4      2024-01  Metropolitan Police Service  Violence and sexual offences
...      ...      ...      ...
1048570  2024-12  Metropolitan Police Service      Vehicle crime
1048571  2024-12  Metropolitan Police Service      Vehicle crime
1048572  2024-12  Metropolitan Police Service  Violence and sexual offences
1048573  2024-12  Metropolitan Police Service  Violence and sexual offences
1048574  2024-12  Metropolitan Police Service  Violence and sexual offences
```

```
      Last outcome category \
0      Unable to prosecute suspect
1      Status update unavailable
2      Status update unavailable
3      Status update unavailable
4      Status update unavailable
...      ...
1048570  Investigation complete; no suspect identified
1048571  Investigation complete; no suspect identified
1048572      Unable to prosecute suspect
1048573  Investigation complete; no suspect identified
1048574  Investigation complete; no suspect identified
```

```
      LSOA name      Location      Latitude \
0      Arun 001C      On or near Queen Street  50.853155
1      Arun 003D      On or near Northfields Lane  50.848940
2      Arun 015C      On or near Kingsmead  50.787270
3      Ashford 005G      On or near Parking Area  51.147655
4      Ashford 010G      On or near Southdown Close  51.110654
...      ...      ...      ...
1048570  Barking and Dagenham 006B      On or near Petrol Station  51.560265
1048571  Barking and Dagenham 006B      On or near Newbury Close  51.558189
1048572  Barking and Dagenham 006B      On or near Parking Area  51.559681
1048573  Barking and Dagenham 006B      On or near Braintree Road  51.556051
1048574  Barking and Dagenham 006B      On or near Stansgate Road  51.556420
```

```
      Longitude
0      -0.552328
```

```

1      -0.666558
2      -0.657460
3       0.876815
4       0.884060
...
1048570  0.148303
1048571  0.151682
1048572  0.155172
1048573  0.150557
1048574  0.151556

```

```
[1048575 rows x 8 columns]
```

```
[3]: print(f"Dataset shape: {df1.shape}")
      print(f"Columns: {df1.columns.tolist()}")
```

```

Dataset shape: (1048575, 8)
Columns: ['Month', 'Reported by', 'Crime type', 'Last outcome category', 'LSOA
name', 'Location', 'Latitude', 'Longitude']

```

**Checking and Removing NA values:** Here you can see that I have dropped the NA values for Latitude, Longitude and LSOA name, this is due to us constructing a scatter diagram showing the types of crime using the Latitude and Longitude, along with choropleth maps that would use the LSOA name, therefore any missing/NA values have to be removed.

```
[4]: print(df1.isnull().sum())
```

```

Month                0
Reported by          0
Crime type           0
Last outcome category 216566
LSOA name            673
Location             0
Latitude             672
Longitude            672
dtype: int64

```

```
[5]: df = df1.dropna(subset=['Latitude', 'Longitude', 'LSOA name'])
      print(df.isnull().sum())
```

```

Month                0
Reported by          0
Crime type           0
Last outcome category 216566
LSOA name            0
Location             0
Latitude             0

```

Longitude 0  
dtype: int64

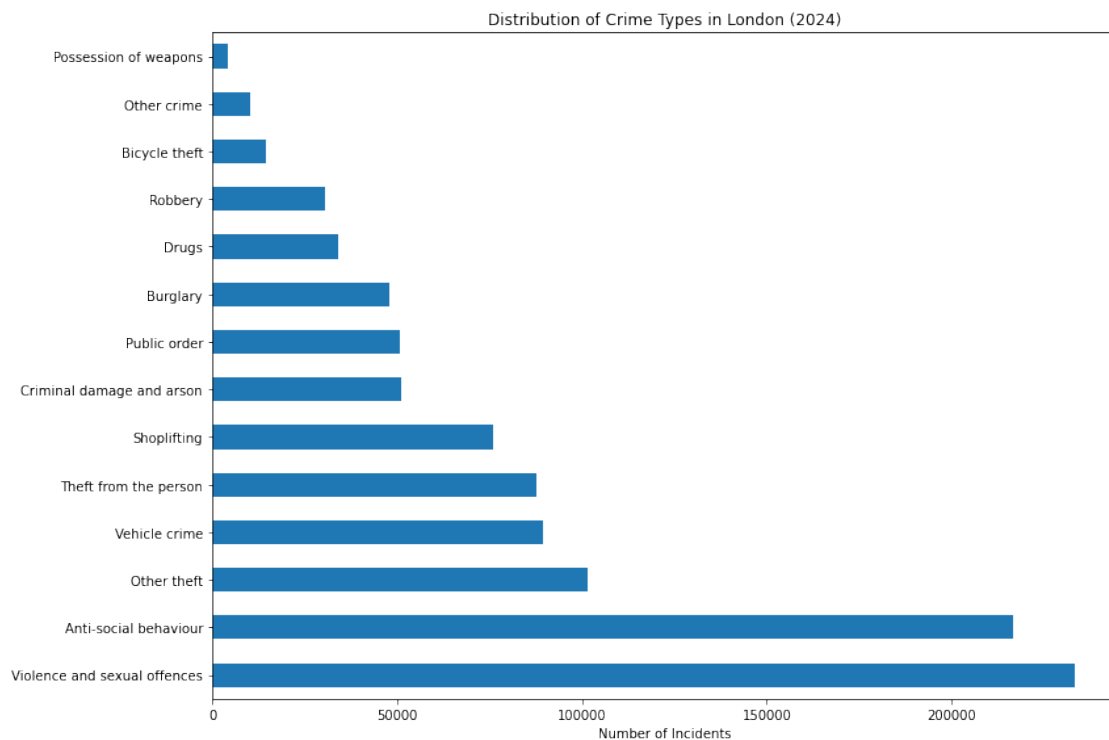
```
[6]: gdf = gpd.GeoDataFrame(  
      df1,  
      geometry=gpd.points_from_xy(df1['Longitude'], df1['Latitude']),  
      crs="EPSG:4326"  
    )  
len(gdf)
```

[6]: 1048575

### 0.1.3 3. Exploratory Data Analysis

#### Part 1: Crime Types Distribution

```
[7]: plt.figure(figsize=(12, 8))  
crime_counts = gdf['Crime type'].value_counts()  
crime_counts.plot(kind='barh')  
plt.title('Distribution of Crime Types in London (2024)')  
plt.xlabel('Number of Incidents')  
plt.tight_layout()  
plt.show()
```

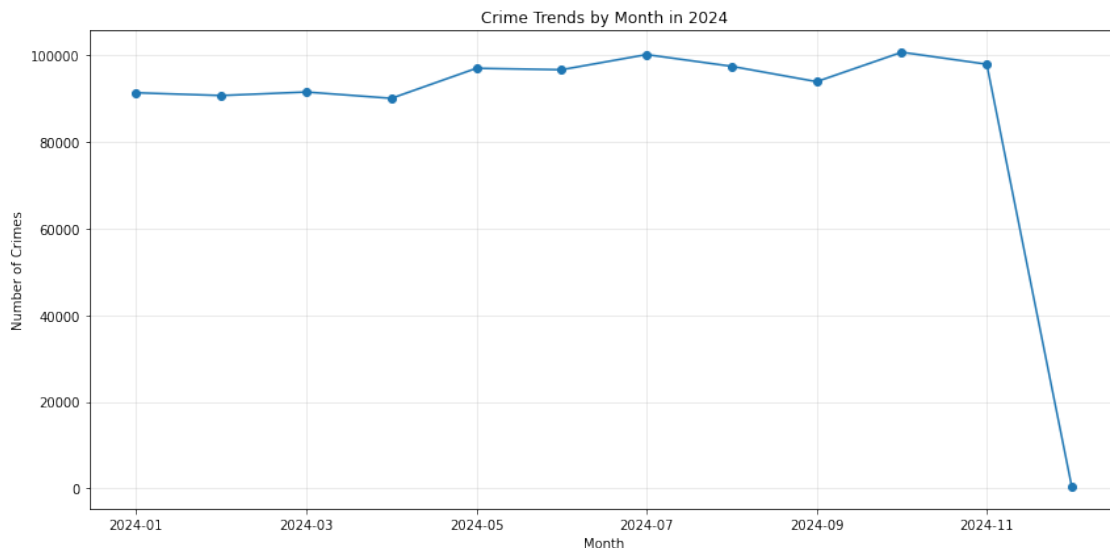


Looking at the distribution for the type of crimes we can see that the 2 most common types of crime are 'Violence and sexual offences', and 'Anti-social behaviour'. And the 2 least common types of crime are 'Possession of weapons' and 'Other crime'.

Below we can look at the the locations of where the most crimes occur.

## Part 2: Crimes by Month

```
[8]: plt.figure(figsize=(12, 6))
monthly_crimes = gdf['Month'].value_counts().sort_index()
monthly_crimes.plot(marker='o')
plt.title('Crime Trends by Month in 2024')
plt.ylabel('Number of Crimes')
plt.xlabel('Month')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Looking at the line graph we can see that October has the highest number of crimes closely followed by July, whereas December has the least number of crimes.

### 0.1.4 4. Geospatial Analysis

#### Part 1: A Basic Crime Map

```
[9]: crime_types = ['Violence and sexual offences', 'Other crime', 'Bicycle theft',
↳ 'Robbery', 'Drugs', 'Burglary', 'Public order',
    'Criminal damage and arson', 'Shoplifting', 'Theft from the
↳ person', 'Vehicle crime', 'Other theft',
    'Anti-social behaviour']
```

```

crime_df = df[df['Crime type'].isin(crime_types)].dropna(subset=['Latitude',
↳ 'Longitude'])

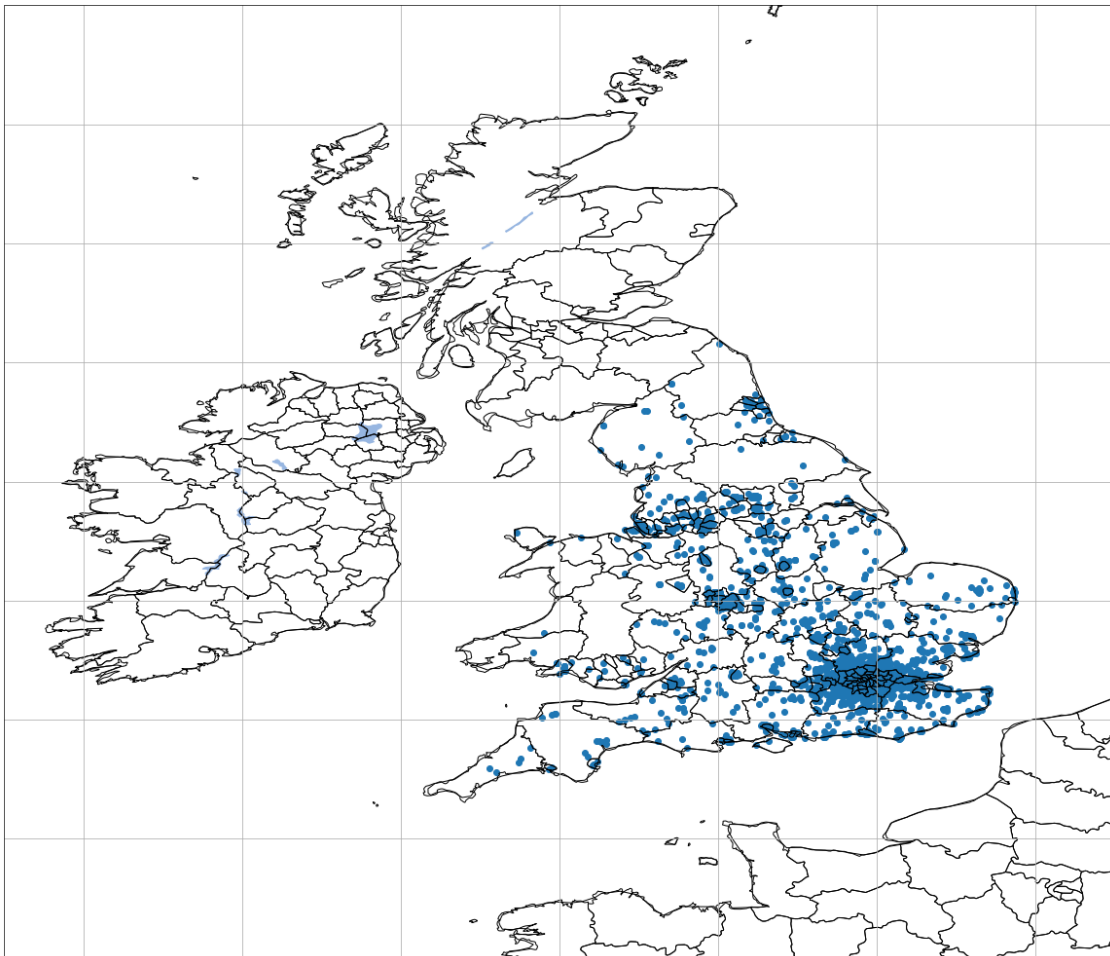
lat = crime_df['Latitude']
lon = crime_df['Longitude']
extant = [-11, 3, 48, 60]
fig=plt.figure(figsize=(24,18))
ax = plt.axes(projection=ccrs.PlateCarree())

ax.set_extent(extant)
ax.gridlines()
ax.coastlines(resolution='50m')
ax.add_feature(cf.LAKES)
ax.add_feature(cf.STATES)

plt.scatter(crime_df['Longitude'],crime_df['Latitude'])

```

[9]: <matplotlib.collections.PathCollection at 0x7f9e904cf370>



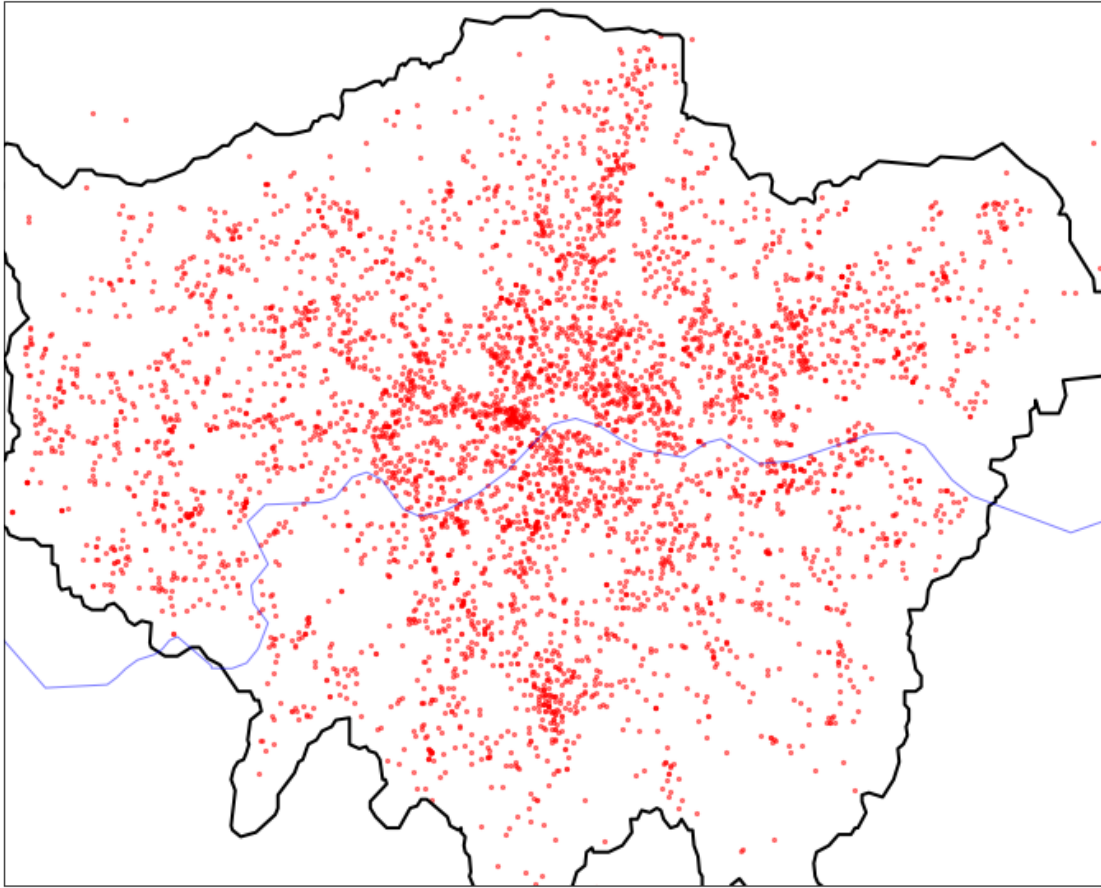
Here there is a crime map of the entire UK, it shows all crime types.

```
[10]: crime_type = 'Violence and sexual offences'
crime_df = df[df['Crime type'] == crime_type]
lats = crime_df['Latitude'].values
lons = crime_df['Longitude'].values

[11]: with open('greater_london.txt') as f:
        london = json.load(f)
        multi_poly = london['features'][0]['geometry']
        london_shape = shape(multi_poly)
        fig = plt.figure(figsize=(12, 10))
        ax = plt.axes(projection=ccrs.Mercator())
        ax.set_extent([-0.5, 0.3, 51.30, 51.70], crs=ccrs.PlateCarree())
        ax.add_geometries([london_shape], crs=ccrs.PlateCarree(),
                           edgecolor='black', facecolor='none', linewidth=2,
                           ↪label='London boundary')
        ax.coastlines('10m')
        ax.add_feature(cfeature.RIVERS.with_scale('10m'), linewidth=1, alpha=0.5,
                           ↪color='blue')
        ax.add_feature(cfeature.LAND.with_scale('10m'), facecolor='white')
        sample_size = min(len(lats), 5000)
        idx = np.random.choice(np.arange(len(lats)), sample_size, replace=False)
        ax.scatter(lons[idx], lats[idx], color='red', s=5, alpha=0.5, transform=ccrs.
                           ↪PlateCarree(), label='Incidents')
```

```
[11]: <matplotlib.collections.PathCollection at 0x7f9e90470640>
```

```
/cvmfs/cybergis.illinois.edu/software/conda/cybergisx/python3-0.8.0/lib/python3.
8/site-packages/cartopy/mpl/style.py:76: UserWarning: facecolor will have no
effect as it has been defined as "never".
```



Here i have plotted a scatter diagram showing the crime hotspots for 'Violence and sexual offences' crimes, and below I have plotted the least common crime type, although they look similar from a distance, if you look closely you can tell that there is a difference in the scatter of the crimes.

```
[12]: crime_type1 = 'Possession of weapons'
      crime_df1 = df[df['Crime type'] == crime_type1]
      lats = crime_df1['Latitude'].values
      lons = crime_df1['Longitude'].values
```

```
[13]: fig = plt.figure(figsize=(12, 10))
      ax = plt.axes(projection=ccrs.Mercator())
      ax.set_extent([-0.5, 0.3, 51.30, 51.70], crs=ccrs.PlateCarree())
      ax.add_geometries([london_shape], crs=ccrs.PlateCarree(),
                        edgecolor='black', facecolor='none', linewidth=2,
                        ↪label='London boundary')
      ax.coastlines('10m')
      ax.add_feature(cfeature.RIVERS.with_scale('10m'), linewidth=1, alpha=0.5,
                        ↪color='blue')
```

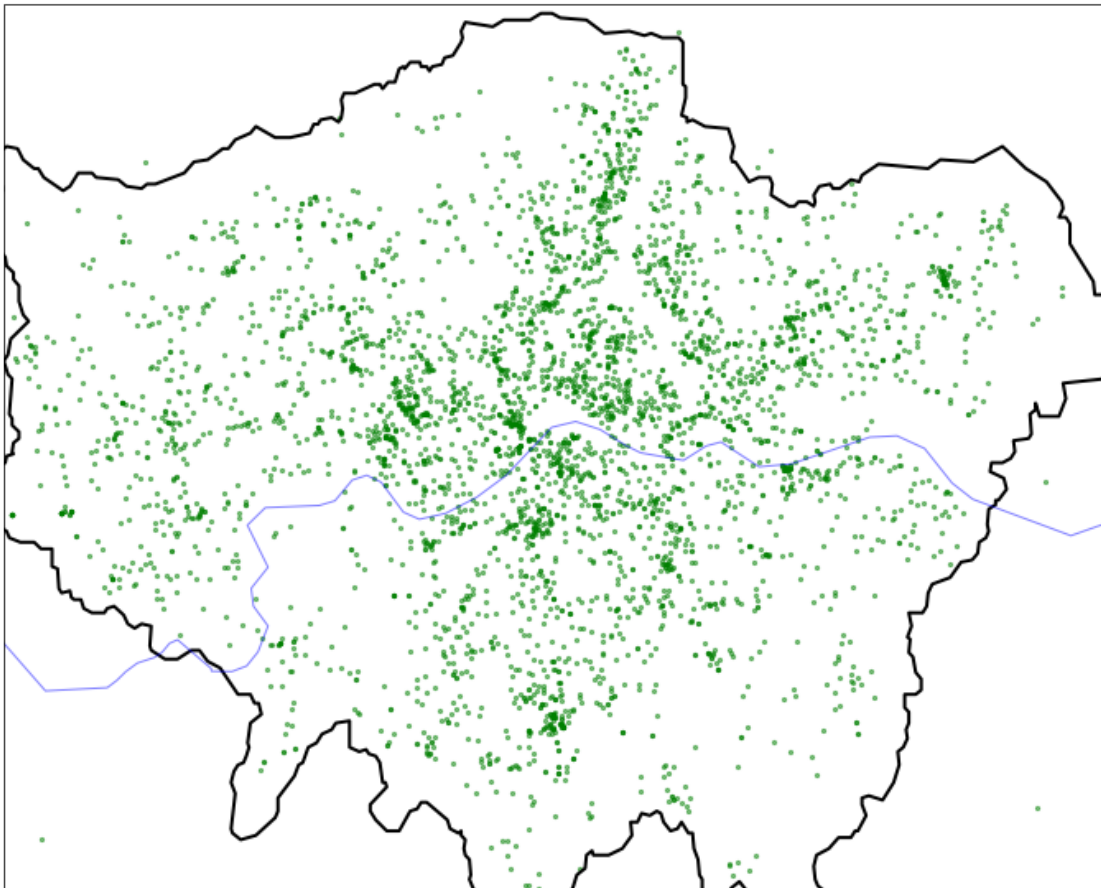


```

ax.add_feature(cfeature.LAND.with_scale('10m'), facecolor='white')
sample_size = min(len(lats), 5000)
idx = np.random.choice(np.arange(len(lats)), sample_size, replace=False)
ax.scatter(lons[idx], lats[idx], color='green', s=5, alpha=0.5, transform=ccrs.
    ↳PlateCarree(), label='Incidents')

```

[13]: <matplotlib.collections.PathCollection at 0x7f9e7162d160>



## Part 2: Choropleth Map

```

[14]: crime_by_area = df1.groupby('LSOA name').size().reset_index(name='crime_count')
crime_by_area['borough'] = crime_by_area['LSOA name'].str.
    ↳extract(r'^([A-Za-z\s]+)')[0]
borough_crimes = crime_by_area.groupby('borough')['crime_count'].sum().
    ↳reset_index()
print("Top 10 boroughs by crime count:")
print(borough_crimes.nlargest(10, 'crime_count'))

```

Top 10 boroughs by crime count:

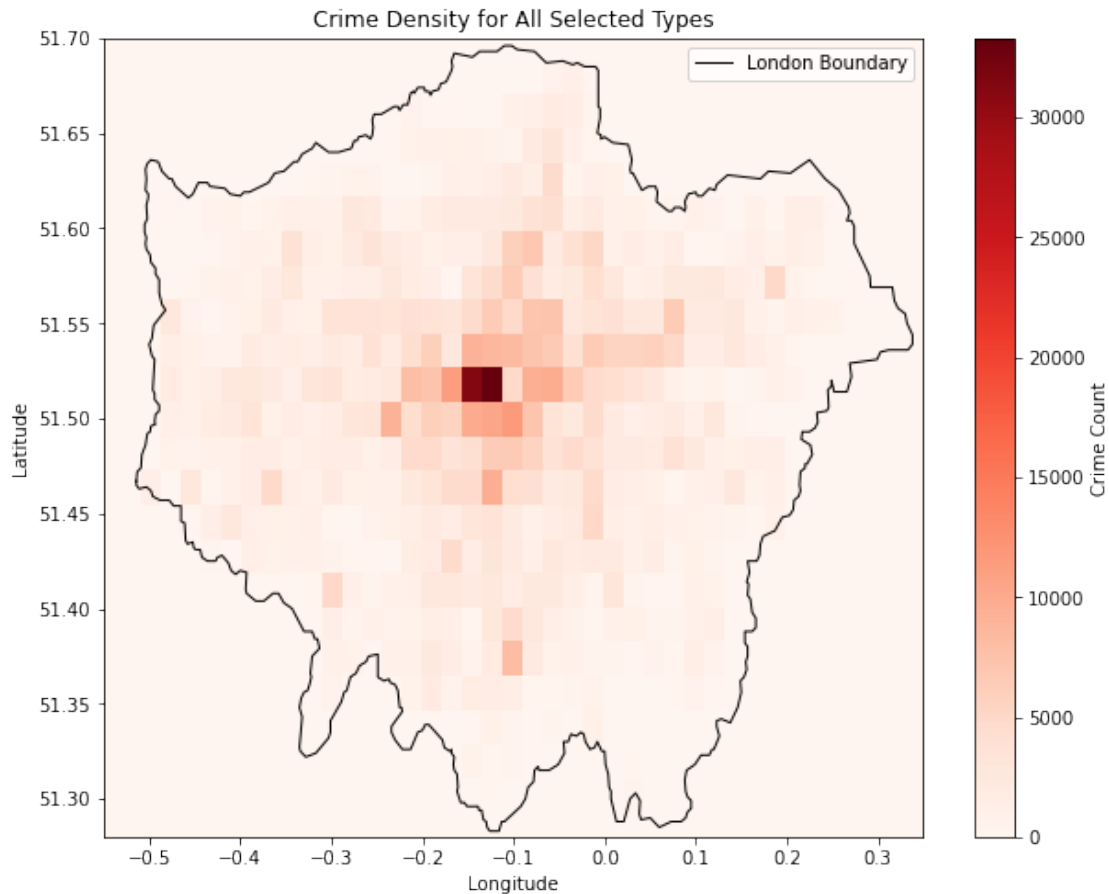
|     | borough       | crime_count |
|-----|---------------|-------------|
| 288 | Westminster   | 92854       |
| 43  | Camden        | 46674       |
| 167 | Newham        | 45399       |
| 138 | Lambeth       | 44055       |
| 235 | Southwark     | 44005       |
| 266 | Tower Hamlets | 42797       |
| 68  | Croydon       | 40071       |
| 28  | Brent         | 39207       |
| 78  | Ealing        | 38563       |
| 109 | Hackney       | 37445       |

```
[15]: crime_types = ['Violence and sexual offences', 'Other crime', 'Bicycle theft',
    ↪ 'Robbery', 'Drugs', 'Burglary', 'Public order',
    ↪ 'Criminal damage and arson', 'Shoplifting', 'Theft from the
    ↪ person', 'Vehicle crime', 'Other theft',
    ↪ 'Anti-social behaviour']
crime_df = df[df['Crime type'].isin(crime_types)].dropna(subset=['Latitude',
    ↪ 'Longitude'])

lat = crime_df['Latitude']
lon = crime_df['Longitude']

with open('greater_london.txt') as f:
    gj = json.load(f)

coords = np.array(gj['features'][0]['geometry']['coordinates'][0][0])
lons_boundary = coords[:, 0]
lats_boundary = coords[:, 1]
plt.figure(figsize=(10, 8))
plt.hist2d(lon, lat, bins=[300, 300], cmap='Reds')
plt.plot(lons_boundary, lats_boundary, color='black', linewidth=1,
    ↪ label='London Boundary')
plt.colorbar(label='Crime Count')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('Crime Density for All Selected Types')
plt.legend()
plt.xlim(-0.55, 0.35)
plt.ylim(51.28, 51.70)
plt.show()
```



This is a heatmap showing the most common areas where all crime occurs.

### Part 3: Crime Density by LSOA

```
[16]: lsoa_crimes = df['LSOA name'].value_counts().reset_index()
lsoa_crimes.columns = ['LSOA name', 'Crime Count']

print("Top 10 LSOA areas by crime count:")
print(lsoa_crimes.head(10))

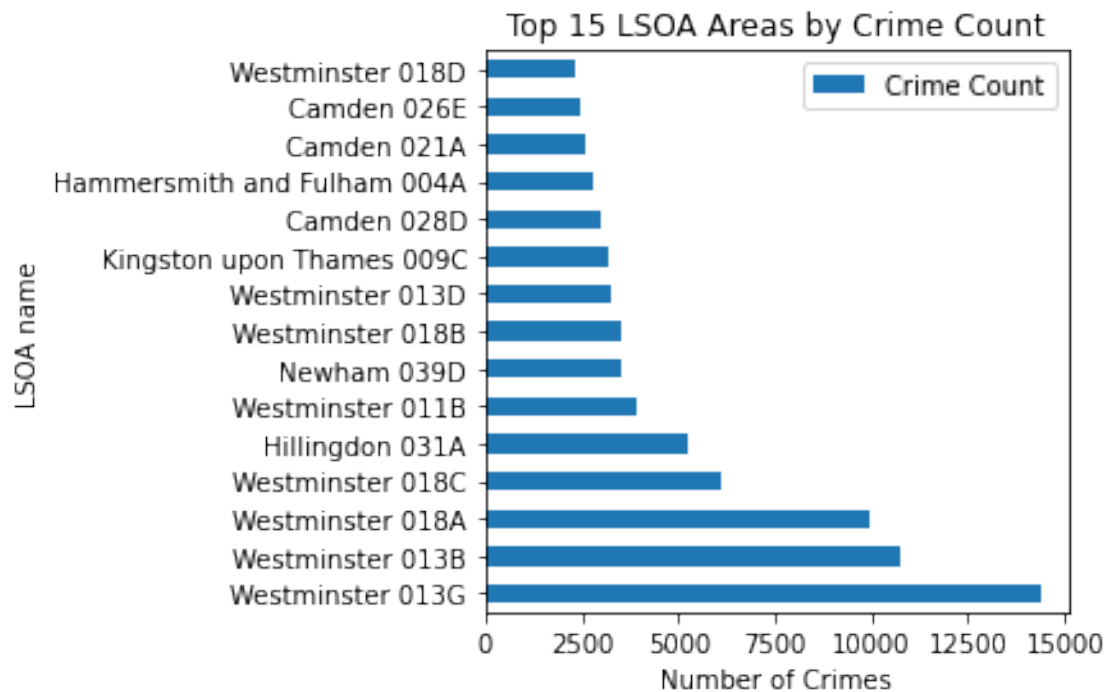
plt.figure(figsize=(12, 8))
lsoa_crimes.head(15).plot(kind='barh', x='LSOA name', y='Crime Count')
plt.title('Top 15 LSOA Areas by Crime Count')
plt.xlabel('Number of Crimes')
plt.tight_layout()
plt.show()
```

Top 10 LSOA areas by crime count:

|   | LSOA name        | Crime Count |
|---|------------------|-------------|
| 0 | Westminster 013G | 14374       |

|   |                           |       |
|---|---------------------------|-------|
| 1 | Westminster 013B          | 10776 |
| 2 | Westminster 018A          | 9920  |
| 3 | Westminster 018C          | 6117  |
| 4 | Hillingdon 031A           | 5265  |
| 5 | Westminster 011B          | 3943  |
| 6 | Newham 039D               | 3542  |
| 7 | Westminster 018B          | 3531  |
| 8 | Westminster 013D          | 3282  |
| 9 | Kingston upon Thames 009C | 3214  |

<Figure size 864x576 with 0 Axes>



Here i have plotted a bar graph showing the crime count by area, and as you can see Westminster has the least number and most number of crimes, dependent on the towns.

### 0.1.5 5. Crime by Loaction.

```
[17]: plt.figure(figsize=(15, 12))
location_cols = ['Borough', 'LSOA_name', 'Location']
location_col = None
for col in location_cols:
    if col in df.columns:
        location_col = col
        break
```

```

if location_col:
    location_counts = df[location_col].value_counts().head(15)

    plt.subplot(2, 2, 1)
    plt.barh(location_counts.index, location_counts.values, color='lightgreen',
edgecolor='black')
    plt.title('Top 15 Areas by Crime Count', fontsize=14, fontweight='bold')
    plt.xlabel('Number of Crimes')
    plt.tight_layout()

```

